

Universidad de los Andes
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas y Computación
Tecnología e Infraestructura de Cómputo - ISIS 1311
Laboratorio 6 - Programar Con Concurrencia

Este trabajo debe ser elaborado individualmente

Puede hacer preguntas al asistente docente y a los monitores para resolver dudas. También puede intercambiar ideas con sus compañeros, pero no puede intercambiar con ellos ningún tipo de código.

> **Tenga en cuenta que los laboratorios están diseñados para apoyar su proceso de aprendizaje e identificar y resolver sus dudas sobre los temas estudiados, antes del parcial.**

Objetivos

- Familiarizarse con los conceptos básicos de programación concurrente en Java: hilos (Thread), ciclo de vida, start(), run(), join() y sleep().
- Identificar y reproducir una condición de carrera en una variable compartida (static) y corregirla mediante exclusión mutua (synchronized).
- Implementar y comparar una solución sencuencial y una concurrente para explicar las diferencia de rendimiento.

Contexto

La concurrencia permite que un programa realice varias tareas “al mismo tiempo” o de forma solapada, aprovechando mejor el CPU y tiempos de espera (E/S). Es fundamental en aplicaciones con alta demanda de procesamiento o que requieren buena capacidad de respuesta (p. ej. servidores, procesamiento masivo de datos, interfaces reactivas). No obstante, la concurrencia introduce retos como condiciones de carrera y contención, que requieren sincronización adecuada para preservar la correctitud del programa.

Preparación

Asegurese que su entorno de trabajo tenga Java 8+. Además, pueden trabajar en el IDE de su preferencia.

Parte 1: Clase Thread

Un hilo (thread) es una unidad de ejecución dentro de un programa. Por lo que, un proceso puede tener uno o varios hilos que comparten la misma memoria, pero ejecutan instrucciones de forma independiente y concurrente.

Existen dos formas principales de crear hilos en Java:

1) Extender la **clase Thread**. (En este taller usaremos esta forma)
2) Implementar la **interfaz Runnable**.

En la clase Thread encontramos los siguientes métodos principales para el manejo de los hilos:

- Método **run()**, contiene el código que ejecutará el hilo cuando se inicie. No se llama directamente, sino que se ejecuta cuando se invoca start().
- Método **start()**, inicia la ejecución del hilo en paralelo al hilo principal. Internamente crea un nuevo flujo de ejecución y llama a run() automáticamente.
- Método **join()**, permite que el hilo principal espere a que otro hilo termine su ejecución antes de continuar.
- Método **sleep()**, suspende temporalmente el hilo por la cantidad de milisegundos que se indique.

Teniendo en cuenta lo anterior ahora vamos a probarlo en código. Para ello, escriba en un archivo Java la siguiente implementación:

```java
public class HelloThread extends Thread {
    private String name;

    public HelloThread(String name) {
        this.name = name;
    }

    @Override
    public void run() {
        System.out.println("Hola desde " + name);
    }

    Run | Debug
    public static void main(String[] args) throws InterruptedException {

        Thread[] threads = new Thread[5];

        for (int i=0; i<5; i++) {
            threads[i] = new HelloThread("T" + (i + 1));
            threads[i].start();
        }

        for (int i=0; i<5; i++) {
            threads[i].join();
        }
        System.out.println(x:"Hilo Principal terminado");
    }
}
```

Figura 1. Clase HelloThread.java.

Ejecute el programa varias veces y responda las siguientes preguntas (en forma de comentarios en el archivo):

- ¿Por qué aparece la anotación **@Override** encima de la definición de run()?
- ¿Por qué el orden de los **println** de los hilos cambia en cada ejecución?

A continuación, modifique el programa para que, usando **Thread.sleep(…)** dentro de run(), la salida siempre muestre los hilos en orden ascendente (primero el hilo 1, luego el hilo 2, luego el hilo 3, etc.).

Explique brevemente (en forma de comentarios en el archivo):

- Que cambio implementó y por qué esa solución hace que la salida quede ordenada.
- Cuál cree que es la desventaja de usar sleep() para ordenar la ejecución de hilos.

Parte 2: Static y Condición de Carerra

En Java, la palabra reservada reservada **static** indica que un atributo o método pertenece a la clase, y no a una instancia escpecífica de ella. Esto significa que si varios objetos (o hilos) acceden a un mismo atributo estático, **todos están compartiendo exactamente la misma variable en memoria**.

En el contexto de concurrencia, esto implica que todos los hilos pueden leer y escribir simultáneamente el mismo recurso compartido, lo que puede generar resultados inconsistentes o impredecibles.

```
private static int contador = 0;
```

Figura 2. Ejemplo de variable compartida en java.

Cree un nuevo archivo/clase llamado **CountThread.java** e implemente un programa que:

1. Declare un atributo estático de tipo entero llamado contador, que sea compartido entre todos los hilos.
2. Defina y ejecute en el método main, cuatro hilos, cada uno de los cuales debe:
    - Incrementar el valor del contador 5000 veces en un bucle (estó debe ir en el método run).
    - Finalizar su ejecución una vez completado el incremento.
3. Al finalizar todos los hilos, el programa debe imprimir el valor final del contador.

Ejecute el programa varias veces y responda las siguientes preguntas (en forma de comentarios en el archivo):

- ¿Cuál deberiá ser el resultado final del contador si no hubiera interferencia entre los hilos?
- ¿Qué valor obtiene realmente al ejecutar el programa varias veces?
- ¿Por qué el resultado no es el correcto? Responda definiendo el concepto de condición de carrera y explíquela con base en el comportamiento observado.

Parte 3: Exclusión Mutua

En la parte anterior observamos cómo, al trabajar con una variable compartida (static), varios hilos pueden modificarla simultáneamente y producir resultados incorrectos. Esto sucede porque las operaciones sobre el recurso no están sincronizadas.

Para resolver este tipo de problemas, existen diferentes formas de manejar la sincronización, una de ellas es la **exclusión mutua**, cuyo objetivo es garantizar que solo un hilo a la vez pueda ejecutar una determinada sección de código llamada **sección crítica**.

En Java, una de las formas más sencillas y comunes de implementar exclusión mutua es mediante la palabra reservada **synchronized**, que puede aplicarse:

1) A un método completo, para que solo un hilo pueda ejecutarlo al mismo tiempo.

```
private synchronized static void incrementar() {
    contador++;  // Sección crítica
}
```

Figura 3. Primera forma de usar synchronized.

2) A un bloque de código, especificando el objeto sobre el que se sincroniza.

```
private static void incrementar() {
    synchronized (CountThread.class) {
        contador++;  // Sección crítica
    }
}
```

Figura 4. Segunda forma de usar synchronized.

De esta forma, se asegura que los hilos accedan de manera ordenada al recurso compartido, evitando interferencias.

A partir del programa desarrollado en la Parte 2, modifíquelo para resolver el problema de condición de carrera utilizando exclusión mutua.

Debe:

1. Sincronizar el acceso a la variable compartida contador, de modo que nunca sea modificada simultáneamente por más de un hilo.
2. Verificar mediante varias ejecuciones que el resultado final del contador sea siempre el correcto.

Responda las siguientes preguntas (en forma de comentarios en el archivo):

- ¿Qué cambio realizó para evitar la condición de carrera?
- ¿Qué significa “sección crítica” y por qué es importante protegerla?

Parte 4: Ejercicio práctico

Implemente un programa que reciba una matriz de enteros de tamaño (n x m) de tipo arreglo de arreglo de enteros. El objetivo es calcular la suma total de todos los elementos de la matriz.

Para ello, primero realice una versión secuencial del programa (sin hilos) que realice esa tarea, puede guardar este programa en un archivo/clase llamado **SumMatrixSequential.java**.

Luego, realice una segunda versión utilizando mútiples hilos (Threads) donde:

1. El programa aparte de recibir la matriz también debe recibir una cantidad de Threads fija.
2. Cada Thread debe encargarse de procesar un conjunto de filas y calcular su suma parcial. Tenga en cuenta que se debe determinar cuantas y cuales filas esta encargado cada hilo de procesar, la idea es que se distribuya de forma optima la carga entre los hilos.
3. Los resultados parciales de cada Thread se deben acumular en un contador total compartido, que debe estar sincronizado utilizando exclusión mutua.
4. El hilo principal debe esperar a que todos los hilos terminen su ejecución antes de imprimir el resultado final.
5. Puede guardar este segundo programa en un archivo/clase llamado **SumMatrixConcurrent.java**.

Una vez tenga ambas versiones funcionando correctamente, cree un tercer archivo llamado **SumMatrixComparison.java** que compare los tiempos de ejecución en milisegundos utilizando matrices de diferente tamaño.

Luego, analice las diferencias de rendimiento entre el enfoque secuencial y el concurrente, y en forma de comentarios escriba una breve conclusión de ese análisis. En particular debe evaluar los siguientes escenarios:

- **Matrices pequeñas:** Ejecute el programa con matrices pequeñas (Ej. 3x3, 5x5, 10x10, etc.). Compare el tiempo entre la versión secuencial y la concurrente, y escriba su conclusión sobre cuál resulta más eficiente y por qué.
- **Matrices grandes:** Ejecute el programa con matrices grandes (Ej. 1000x1000 o mayores). Analice si la versión concurrente presenta una mejora significativa en el tiempo de ejecución respecto a la secuencial, y explique por qué sucede esto.
- **Matrices grandes con demasiados threads:** Ejecute el programa usando una cantidad de hilos muy superior (Ej. Un thread por fila). Observe qué ocurre con el rendimiento y comenté por qué un número excesivo de hilos puede afectar negativamente la ejecución.

Condiciones de entrega

- **Debe entregar en un .zip los archivos solicitados en cada parte del laboratorio junto con los comentarios que respondan a las preguntas propuestas:**
  - HelloThread.java
  - CountThread.java
  - SumMatrixSequential.java
  - SumMatrixConcurrent.java

    - SumMatrixComparison.java
- Política de entrega tarde. Para las entregas tarde, se aplicará la siguiente política: por cada 30 minutos, o fracción, de retraso con respecto a la hora de entrega establecida en Bloque Neón habrá una penalización de 0,5/5.
- La calificación de este laboratorio está condicionado a la asistencia, en caso de no asistir a clase o no firmar la hoja de asistencia **su nota será 0**.